Understanding the Switching Bayesian Observer Code

Understanding the Switching Bayesian Observer code (and porting it to Python)

This document explains how the MATLAB code in `projInference-gh-pages/` works, what it actually implements versus what the paper claims, and exactly what you need to do to reimplement it in Python/Jupyter so you can build **switching (MAP)**, **online/sequential**, and **sampling** observers. It is the companion to the runnable notebook `Switching_Bayesian_Observer_starter.ipynb`.

The code accompanies Laquitaine & Gardner (2018), *A Switching Observer for Human Perceptual Estimation*. Subjects estimate the direction of a noisy motion stimulus; the "trick" is that stimulus directions are drawn from a prior (a von Mises centred on 225°) whose width changes across blocks, and optimal behaviour requires exploiting that prior. The model asks which readout rule (peak, mean, or sample of the posterior) best explains how people trade off the sensory evidence against the learnt prior.

The one-paragraph model

On each trial the true direction is `d`. The brain forms a noisy sensory measurement `m ~ vonMises(d, k_llh)`, where the concentration `k_llh` encodes stimulus reliability (higher motion coherence → larger `k_llh`). It multiplies the likelihood of that measurement by a learnt prior `vonMises(225°, k_prior)` (and optionally a fixed "cardinal" prior at 0/90/180/270°) to get a posterior over directions. A **readout rule** collapses the posterior to a single reported estimate: its **peak** (MAP — this is the "switching" observer), its **mean** (BLS), or a **random draw** (Sampling). The estimate is then blurred by motor noise `vonMises(0, k_motor)`, and on a fraction `p_rand` of trials it is replaced by a uniform random guess (a lapse term). Fitting the model = finding the parameters that maximise the likelihood of the observed estimates.

The full parameter set is: three likelihood concentrations `k_llh` (one per coherence), four prior concentrations `k_prior` (one per prior width), an optional cardinal-prior strength `k_cardinal`, a lapse rate `p_rand`, a motor-noise concentration `k_motor`, and an optional heavy-tail weight `weight_tail`.

What the code actually contains (important)

The repository is titled "A Switching Bayesian observer," but the switching observer is **not a separate model file** — it is simply the **MAP readout** of the one Bayesian estimator that is implemented. Concretely:

The estimator `SLGirshickBayesLookupTable.m` supports two readouts inline, selected by a string argument: `MAPReadout` (the switching observer) and `BLSReadout` (posterior mean). MAP produces the switching behaviour because the argmax of a bimodal posterior jumps discretely between the prior mode and the likelihood peak; when the posterior is genuinely bimodal the function returns *two* equiprobable percepts, and that is the discrete "switch."

The **Sampling** readout is *referenced* — `SLgetLoglBayesianModel.m` calls a function named `SLBayesSamplingLookupTable` — but **that file is not present in this repository.** So a sampling observer has to be reconstructed. The notebook does this (§3): the sampling observer's estimate distribution is just the posterior marginalised over measurements, `P(e|d) = Σ_m post(e|m)·P(m|d)`, which is a one-line matrix multiply.

There is **no online/sequential model anywhere** in the code. Every prior is fixed per block: the block's `prior_std` selects one of four `k_prior` parameters. An online observer that updates its prior trial-by-trial is entirely new work — the notebook gives you a runnable scaffold for it (§8).

How the MATLAB code is organised (call graph)

`SLfitBayesianModel.m` is the ~3,400-line top-level driver. Most of its length is argument parsing, data-loading branches, folder bookkeeping, and plotting — not model math. It dispatches to one of a few "analyses" selected by string flags: `MaxLikelihoodFit` (fit parameters), `CrossValR2Fit`, `modelPredictions` (simulate and plot mean/std/distributions), and `stdBestfitP`. The maximum-likelihood path is the nested function `MLft5`, which builds ~10 sets of initial parameters (an 8×-stronger/weaker grid over likelihood and prior strengths) and runs Nelder–Mead (`fminsearch`) from each — multiple restarts because the MAP likelihood surface is multimodal.

The actual computation lives in three files, and these are the only ones you need to port:

`SLgetLoglBayesianModel.m` is the objective function. It builds, per experimental condition, the estimate-distribution lookup table; assembles a per-trial matrix `P(estimate | trial)`; mixes in the lapse term `(1−p_rand)·P_bayes + p_rand/360`; convolves with motor noise; and returns `−Σ log P(observed estimate)` (plus AIC). This is what the optimiser minimises.

`SLGirshickBayesLookupTable.m` is the estimator itself — the heart of the model. Given a condition's `(k_llh, prior mode, k_prior, …)` it returns the matrix `L[e, d] = P(report estimate e | true direction d)` over the integer grid 1…360°. It computes the measurement distribution, the posterior for every possible measurement, applies

the readout, and marginalises. It includes a numerical closed-form fix (Murray & Morgenstern 2010) for when a very sharp prior makes the naïve posterior underflow to all-zeros.

`vmPdfs.m` is the von Mises pdf generator, written for numerical stability at very large concentration `k` (sharp priors) using the exponentially-scaled Bessel function `besseli(0,k,1)`. Everything else in `codes/assets/` (dozens of `SL*` files) is plotting, circular-statistics helpers, and data wrangling — you do **not** need to port those; NumPy/pandas/Matplotlib replace them.

Data flow: raw `.mat` files → `SLMakedatabank.m` builds a "databank" struct → the fit reads columns `estimatedFeature`, `FeatureSample` (true direction), `StimStrength` (coherence), `Pstd` (prior width), `priormodes`. Crucially, the repo already ships a flattened export at `data/csv/data01_direction4priors.csv`, so **you can skip the entire MATLAB data-loading layer** and read the CSV directly in pandas.

The math, in the order the code computes it

The direction space is discretised to the 360 integer degrees, so every distribution is a length-360 vector and Bayesian integration is elementwise multiplication followed by normalisation. For one condition:

The **measurement distribution** `P(m | d)` is `vonMises(d, k_llh)` — one column per displayed direction. The **likelihood** `P(d | m)` over the 360 hypotheses given each of the 360 possible measurements is the same von Mises family transposed. The **posterior** for each measurement is `likelihood × prior` (optionally `× cardinal prior`), column-normalised. The MATLAB code rounds the posterior to ~6 decimal places before taking the argmax, so that round-off noise doesn't create spurious modes — the notebook reproduces this.

The **readout** maps each measurement's posterior to an estimate. MAP takes the argmax (possibly two values when bimodal). BLS takes the circular mean. Sampling reports a draw (so its estimate distribution equals the posterior). Then the code **marginalises over measurements** to get the estimate distribution per true direction: `P(e | d) = Σ_m P(e | m)·P(m | d)`, where `P(e | m) = 1/(number of tied percepts)`.

Finally, at the trial level, the lapse mixture and a **circular convolution** with motor noise `vonMises(0, k_motor)` turn `P(e|d)` into `P(estimate | model)`, and the log-likelihood reads off the probability of the actually reported estimate. Estimates equal to 0° are folded to 360°; probabilities are floored at 1e−320 to avoid `log(0)`.

MATLAB → Python: will it port? (assessment)

Yes — the model ports cleanly, and in fact becomes much shorter, because the parts that make the MATLAB code long (string-flag dispatch, `.mat` loading, bespoke plotting

and circular-stats helpers) all collapse into standard Python libraries. The concrete mapping:

`vmPdfs.m` → `scipy.special.i0e` gives you the same stable von Mises; ~10 lines. `SLGirshickBayesLookupTable.m` → ~40 lines of NumPy (the notebook's `girshick_lookup`). `SLgetLoglBayesianModel.m` → the notebook's `trial_loglike`. `fminsearch`/`MLft5` → `scipy.optimize.minimize(method="Nelder-Mead")` with several random restarts. Data loading → one `pandas.read_csv`. Plotting → Matplotlib.

Things to watch — the genuine porting gotchas:

The reported estimate in the CSV is stored as a 2-D vector `(estimate_x, estimate_y)`, not an angle; convert with `atan2(y, x)` and wrap to 1…360. Indexing is off-by-one: MATLAB is 1-based and uses direction 360 (not 0), so keep a 1…360 grid and subtract 1 only when indexing NumPy arrays. Circular convolution must wrap around 360° — use FFT-based convolution (the notebook does), not `numpy.convolve`. The MAP tie-handling (multiple equiprobable percepts) is essential to the switching behaviour and easy to drop by accident — keep it. Reproduce the posterior-rounding step or you'll get extra spurious modes. And keep the lapse term and the 1e−320 floor, or occasional unpredicted estimates will send the log-likelihood to −∞. The motion-energy variant of the model loads precomputed sensory-likelihood `.mat` files (`likelihood006.mat`, etc.) that are not needed for the standard von Mises model and can be ignored.

None of these is a blocker. There is no MATLAB-specific dependency in the core math — it is all von Mises pdfs, elementwise products, normalisation, argmax/mean, and one circular convolution.

How the notebook maps to all this, and your three target models

The notebook `Switching_Bayesian_Observer_starter.ipynb` implements `vm_pdf`, `girshick_lookup` (MAP and BLS), `sampling_lookup`, and `trial_loglike`, loads the CSV, reproduces the prior-attraction bias curves, overlays a MAP prediction on real data, and computes a per-trial negative log-likelihood. It runs end-to-end with no MATLAB.

For your three priorities: **Switching** is the MAP readout already in `girshick_lookup(readout="MAP")` — to study switching explicitly, look at trials where it returns two MAP percepts (bimodal posterior); those are the discrete switch events. **Sampling** is `sampling_lookup` in §3, runnable now; if you later find the original `SLBayesSamplingLookupTable.m`, diff it against this marginal definition. **Online/sequential** is the new-work part: §8 gives a delta-rule scaffold that keeps a running circular mean and concentration of the directions seen so far and feeds them into the same estimator as a time-varying `(mode, k_prior)`. Replace that rule with your learning hypothesis (Bayesian filtering over prior parameters, a leaky integrator, or a change-point/switching prior) and fit its learning rate by maximum likelihood exactly like any other parameter.

Practical fitting notes carried over from the original: use multiple random restarts (the MATLAB code uses ~10 initial-parameter sets) because the MAP likelihood is multimodal; and cache each condition's lookup table by `(k_llh, k_prior)` rather than recomputing per trial — the tables are trial-independent.